

Guidelines for Submitting Excellent Software for M.Sc. in Computer Science

1. Overview

This document defines the academic and technical requirements for submitting a high-quality software project for an M.Sc. in Computer Science. It provides detailed expectations for documentation, code structure, testing, research presentation, and software quality.

2. Project and Design Documents

2.1 Product Requirements Document (PRD)

Defines the project's purpose, scope, and objectives, including:

- Stakeholders, KPIs, and success criteria.
- Acceptance tests and measurable outcomes.
- Functional and non-functional requirements, user stories, and use cases.
- Scalability, performance, limitations, and dependencies.
- Timeline, milestones, and deliverables.

2.2 Architecture Document

Describes system architecture, diagrams, and design decisions:

- Use of the **C4 model** (Context, Container, Component, Code).
 - UML diagrams and deployment architecture.
 - Architectural Decision Records (ADRs) summarizing trade-offs and alternatives.
 - API definitions, data schemas, and interface contracts.
-

3. Code Documentation and Project Structure

3.1 Comprehensive README

The README serves as the main manual. It must include:

- Installation instructions and environment setup.
- Usage examples and troubleshooting.
- Configuration details and contribution guidelines.
- Licensing, authorship, and references.

3.2 Modular Project Structure

The project must have a clear modular organization with logical separation of concerns (e.g., feature-based or layered architecture). Example structure:

project-root/

├── src/ (source code)

```
| ├── agents/
| ├── utils/
| └── config/
├── tests/
├── data/
├── results/
├── docs/
├── assets/
├── notebooks/
├── README.md
├── requirements.txt
└── .gitignore
```

3.3 Code Quality and Comments

All functions, classes, and modules must include **docstrings** and inline comments. Follow DRY (Don't Repeat Yourself) and single-responsibility principles. Code should be self-explanatory, well-structured, and consistent in naming.

4. Configuration Management and Security

4.1 Configuration Files

Store configuration separately (e.g., .json, .yaml, .env). Never hard-code secrets. Include an example configuration (env.example). Use .gitignore to exclude sensitive files.

4.2 Information Security

Protect API keys and credentials with environment variables or secret-management tools. Apply least-privilege access and rotate secrets periodically.

5. Software Testing and Quality

5.1 Unit Tests

Unit tests must cover **at least 70–80 %** of the codebase. Use frameworks such as pytest or unittest. Automate test execution in CI/CD pipelines and include coverage reports.

5.2 Handling Edge Cases and Errors

Test critical and edge cases. Ensure graceful degradation, defensive programming, and informative error handling.

5.3 Expected Test Outputs

Provide clear test results, success/failure rates, and error logs.

6. Research and Results Analysis

6.1 Parameter Sensitivity Analysis

Analyze how parameter changes affect outcomes using methods like OAT (one-at-a-time) or variance-based sensitivity analysis.

Present findings with visualizations (heatmaps, line charts, etc.).

6.2 Analysis Notebooks

Include **Jupyter Notebooks** or equivalent showing code, metrics, and visualizations.

Document methodology, results, and interpretation clearly.

6.3 Visual Presentation of Results

Provide well-designed visual outputs — bar, line, scatter, and waterfall charts — with captions and explanations for every figure.

7. User Interface (UI) and User Experience (UX)

7.1 Usability Criteria

Evaluate software usability using **Nielsen's 10 Heuristics**, including:

- Visibility of system status
- Match between system and real world
- User control and freedom
- Consistency, error prevention, and minimalist design
- Help, documentation, and recovery from errors

7.2 Interface Documentation

Include screenshots or diagrams illustrating workflows, accessibility features, and interaction flow.

8. Version Control and Development Documentation

8.1 Git Best Practices

Use clear commit messages, meaningful branches, pull requests, and tagging.

Keep a readable commit history.

8.2 Prompt Engineering Log

Document AI prompts used in code generation or development.

For each prompt, record its purpose, context, and results.

9. Costs and Budget

9.1 Cost Analysis

Estimate computational or API token usage costs.

Example cost table (for GPT-4, Claude, etc.) should include input/output tokens and total cost.

9.2 Budget Management

Track costs over time, monitor API usage, and optimize cost-effectiveness.

10. Extensibility and Maintainability

10.1 Extension Points

Design the architecture for extensibility using plugins or middleware.
Define clear interfaces and lifecycle hooks (e.g., beforeCreate, afterUpdate).

10.2 Maintainability

Ensure modular, testable, and reusable code.
Support easy updates, analysis, and replacement of modules.

11. International Quality Standards

11.1 Software Quality Attributes (ISO 25010)

The project should comply with ISO/IEC 25010 standards, covering:

- Functional suitability
- Performance efficiency
- Compatibility and interoperability
- Usability and reliability
- Security and maintainability
- Portability and adaptability

12. Final Submission Checklist

Before submission:

- Include PRD, architecture, and README documents.
- Provide source code with modules, tests, and docstrings.
- Attach configuration examples and .gitignore.
- Verify test coverage $\geq 70\%$, run edge-case tests, and include visual results.
- Document cost, research findings, and extension design.

13. Additional References and Standards

Includes references to **MIT ACIS**, **ISO/IEC 25010**, **Google Engineering Practices**, **Microsoft REST API Guidelines**, and **Nielsen UX Heuristics**.

14. Important Note

This document summarizes best academic and engineering practices for M.Sc. software submissions. Projects are evaluated for depth, rigor, and adherence to modern software-engineering standards, including AI-assisted development practices.

15. English References

A full list of 25 sources from MIT, ISO, Google, Microsoft, Monday.com, Aha!, Codacy, Stack Overflow, and others is provided for best-practice frameworks and templates.

Would you like me to produce a **shorter summary (1 page)** version for inclusion in your thesis or project README?